
PAXLEY

CODE SECURITY PLATFORM

P E R S P E C T I V E S · 2 0 2 6

Why Securing Modern Software the Old Way Risks Obsolescence

Ten shifts modern engineering teams need to make — and how the Paxley Code Security Platform supports each one.

THE SECURITY MODERNISATION TRAP

Modern software is not built the way it was ten years ago. It is assembled from open-source packages, generated with AI assistance, deployed through automated pipelines, packaged into containers, configured through infrastructure-as-code, and released many times a day across cloud environments.

Yet many organisations still try to secure it with practices designed for a slower world: annual penetration tests, late-stage security reviews, manual change boards, spreadsheet-based dependency tracking, and “secure coding” checklists that sit outside the developer workflow. That is the security modernisation trap.

The problem is not that old secure-development practices were wrong. Many were valuable for the era they served. The problem is that they were designed around slower delivery, smaller dependency graphs, fewer deployment paths, and clearer boundaries between development, security, and operations. Modern software breaks those assumptions: a single release may now depend on hundreds of third-party packages, multiple container images, cloud permissions, CI/CD workflows, generated code, infrastructure templates, secrets, feature flags, and external APIs.

The future of software security cannot be an old SDLC with a few scanners bolted on. It has to be a continuous code-trust model: security embedded into repositories, pull requests, pipelines, build systems, release gates, and post-release response. Below are ten areas where traditional practices fall short — and what modern software security now requires instead.

TEN SHIFTS MODERN SECURITY MUST MAKE

1. From Phase-Gate Security to Continuous Security Evidence

The old model treated security as a phase: requirements, design, build, test, security review, release. That model breaks when teams ship continuously and deploy behind feature flags. Modern security needs to generate machine-readable evidence on every pull request, dependency update, image build, infrastructure change, and release candidate.

The question changes from “Did we do a security review before launch?” to “Can we prove, for this release, what code, dependencies, controls, and exceptions were present?” Security becomes less about periodic inspection and more about persistent assurance.

2. From Secure Coding Training to Secure Code Workflows

Annual OWASP training and a PDF of guidelines are not enough. Developers need actionable feedback inside the workflow where the mistake happens — in pull requests, IDEs, commit hooks, and CI checks. SAST, secret scanning, dependency checks, and policy validation should be close enough to the developer to catch insecure patterns before they become backlog items.

The most important design principle is relevance. A pull request should not punish a developer for every historical vulnerability in a five-year-old repository. It should focus on what changed, what risk was introduced, and what fix is practical. Modern secure coding is not just education — it is workflow design.

3. From Manual Code Review to Multi-Layer Code Analysis

Peer review still matters for intent, design, and high-risk decisions, but code volume has exceeded human review capacity — especially with AI-assisted development. Machines should continuously scan for known patterns, unsafe data flows, hard-coded secrets, vulnerable dependencies, IaC misconfiguration, container vulnerabilities, license risk, and maintainability regressions.

The key is not adding more scanners. The key is normalising their output so teams are not forced to reconcile five dashboards, five severity models, and five definitions of “critical.” Modern code security requires orchestration, not tool sprawl.

4. From Dependency Awareness to Supply Chain Governance

“Do we have any vulnerable packages?” is only the start. Modern teams also need to know what components are in this release, which dependencies are direct or transitive, which vulnerabilities are reachable, which packages are abandoned, which licenses are unacceptable, and which components changed between releases.

SBOMs are central here — but only when tied to builds, releases, vulnerability monitoring, VEX statements, customer disclosure, and incident response. A spreadsheet exported once and filed away is not enough. Modern supply-chain security is not a spreadsheet — it is release intelligence.

5. From Secret Cleanup to Secret Prevention

Discovering exposed credentials after the fact is reactive and expensive. Modern secret security requires prevention and containment: detect before commit, block in pull requests, scan across git history, and rotate automatically when exposure is suspected. CI/CD systems should use short-lived credentials and minimum scope.

The deeper lesson is that secret management is part of release management. A pipeline that can deploy to production is a privileged system. Any secret available to it must be treated as production-sensitive.

6. From Application Security to Environment-as-Code Security

A modern release includes Terraform modules, Kubernetes manifests, Helm charts, GitHub Actions workflows, Dockerfiles, IAM policies, ingress rules, and environment variables. Misconfiguration in any of these can lead to a security failure, even when the application code is clean.

The old separation between “application security” and “infrastructure security” becomes fragile when infrastructure is committed to the same repositories and deployed through the same pipelines. In the modern world, “secure code” includes the code that creates the environment.

7. From CI/CD as Automation to CI/CD as a Security Boundary

CI/CD systems pull source code, retrieve dependencies, access secrets, build artifacts, sign packages, publish containers, deploy to environments, update infrastructure, and trigger downstream systems. That makes the pipeline one of the most sensitive parts of the organisation.

Modern release security treats CI/CD as critical infrastructure. Pipeline definitions should be reviewed like production code; workflow changes should require approval; build runners should be isolated; permissions should be least-privilege; external actions should be pinned and verified. The pipeline is not just how software gets released — it is part of what must be secured.

8. From Release Approval to Artifact Integrity

Traditional release management centred on approval: did the change pass testing, did the right people sign off, did the release notes look correct? Modern release management needs all of that — plus provenance.

The central question is not only “Was this release approved?” but “Can we prove this artifact came from the expected source, was built by the expected workflow, used the expected dependencies, and was not tampered with after build?” That requires attestations, signing, and traceability. It shifts release management from trust-by-process to trust-by-evidence.

9. From Change Advisory Boards to Policy-as-Code Release Gates

Manual gates either slow teams down or become ceremonial rubber stamps. Modern governance needs automated policy gates evaluated consistently in CI/CD: block release on a new critical vulnerability,

require remediation for reachable high-risk dependencies, prevent deployment if secrets are detected, require SBOM generation, require signed artifacts.

This does not remove human judgment — it moves it to defining policy, reviewing exceptions, and handling ambiguous risk decisions. The best release gates are not the most restrictive. They are the clearest: fast, explainable, and auditable.

10. From Post-Release Firefighting to Continuous Remediation

New CVEs appear after release. Dependencies change. Containers age. A release that was acceptable yesterday may be risky tomorrow. Continuous remediation means tracking which releases are affected by new vulnerabilities and prioritising them by exploitability, reachability, and business criticality — not raw scanner severity alone.

Low-risk dependency upgrades can become automated pull requests. Known fixes can be suggested directly. Exceptions can expire automatically. The goal is not to eliminate all vulnerabilities instantly — it is to make risk visible, owned, prioritised, and continuously reduced. Modern security is not a one-time hurdle to release. It is an operating model.

THE REAL DEBATE: TOOLS OR A NEW MODEL?

It is tempting to frame this as “old tools versus new tools.” That misses the point. The future of software security is not simply buying more scanners, adopting AI remediation, or generating SBOMs because a customer asked for them. The real debate is whether security practices align with how software is built and released today.

- ▶ *If software is developed continuously, security must be continuous.*
- ▶ *If software is assembled from third-party components, security must include supply-chain governance.*
- ▶ *If infrastructure is code, infrastructure must be scanned like code.*
- ▶ *If CI/CD deploys to production, pipelines must be protected like production.*
- ▶ *If releases are automated, release evidence must be automated.*
- ▶ *If AI increases code volume, security cannot scale by headcount or manual review alone.*

This is why old secure-development practices, applied unchanged, increasingly risk obsolescence. They secure a world of slower releases, clearer perimeters, smaller dependency graphs, and manual control points. Modern software requires a different model: code trust assurance across the full lifecycle.

GETTING STARTED WITHOUT BOILING THE OCEAN

Organisations do not need to transform everything at once. A practical starting point is to map the release path and identify gaps in security evidence. Start with ten questions:

1. Do we know what code, dependencies, containers, and infrastructure definitions are included in each release?
2. Are pull requests scanned for code vulnerabilities, secrets, dependency risk, and infrastructure misconfiguration?
3. Are developers receiving feedback on what changed, or are they inheriting the entire historical backlog?
4. Are SBOMs generated automatically for production releases?
5. Are release artifacts signed, traceable, and linked to source commits and build workflows?
6. Are CI/CD workflows protected by least privilege, review, and controlled secrets?
7. Are third-party actions, packages, and base images pinned or verified?
8. Are security release gates defined as policy, or handled each time manually?
9. Are exceptions time-bound, owned, and visible?
10. Can we prove the security posture of a release after it ships?

The answers will reveal where the old model is still doing useful work — and where it has become a bottleneck or blind spot. Modern software security does not mean abandoning everything learned from secure development. It means operationalising those lessons inside the systems that actually build and release software today.

HOW THE PAXLEY CODE SECURITY PLATFORM ADDRESSES THESE CONCERNS

Paxley was built for the exact software security model described in this article. Each of the ten shifts above maps directly to a capability inside the Paxley Code Security Platform — with code quality and health included as the eighth capability alongside seven security domains.

The table below maps each concern to the Paxley capability that addresses it. The full platform is a unified product with a single dashboard, a single normalised finding model, and a single per-repository price.

#	The Shift Required	How Paxley Delivers
1	Continuous security evidence on every change	Scans run on every pull request, push, scheduled cadence, and ad-hoc upload. Every job produces normalised, machine-readable findings with fingerprints, severity, ownership, and release linkage. Posture history is retained for audit.
2	Secure code workflows inside the PR	Native GitHub App posts diff-aware PR comments focused on what changed. Developers see findings introduced by the current change first; security teams keep the full backlog in the dashboard—VS Code extension on the 2026 roadmap.
3	Multi-layer analysis without tool sprawl	12 scanning engines (Opengrep, CodeQL, OSV-Scanner, govulncheck, Gitleaks, Trivy, KICS, PHPStan, Psalm, PHPMD, plus internal normalisation) orchestrated through one platform. Unified severity model, deduplicated findings, single dashboard.
4	Supply chain governance, not just CVE checks	SCA across direct and transitive dependencies with reachability filtering. SBOM lifecycle (CycloneDX, SPDX, VEX, NTIA fields, diff tracking, historical retention). License policy engine. Component change tracking between releases.
5	Secret prevention, not cleanup	Gitleaks-powered detection across source and full git history. PR blocking on new secret introduction. Audit export for incident response. Customer-runner deployment keeps secret detection within the customer's infrastructure.
6	Environment-as-code security	IaC scanning for Terraform, CloudFormation, Kubernetes manifests, Helm charts, GitHub Actions workflows, Pulumi, CDK, and Bicep via KICS and Trivy. Container image scanning via Trivy. Same dashboard and finding model as code findings.
7	CI/CD as a protected security boundary	Workflow-aware scanning of pipeline definitions and third-party actions. RBAC, SSO/SAML (Okta, Azure AD, Google, JumpCloud), and audit logs on the platform itself. The customer-runner option means that scans execute in your isolated environment.
8	Artifact integrity and provenance	Build job IDs linked to source commits, scan results, and release artifacts. SBOM provenance with cryptographic hashes. SOC 2 Type II audited. Audit-grade reporting that proves what was scanned, what was found, what was accepted.
9	Policy-as-code release gates	The policy engine evaluates blocking conditions in CI: a new critical vulnerability, a reachable high-risk dependency, a detected secret, a missing SBOM, or an unsigned artifact. Suppressions are time-bound, owned, and visible. Gate decisions are explainable to developers.
10	Continuous remediation, not periodic firefighting	AI-assisted remediation guidance with auto-generated fix PRs on Growth tier and above. Reachability-aware prioritisation. Exception expiry. Dashboards show remediation velocity, aging risk, and release exposure. Continuous scanning keeps the picture current.

THREE ARCHITECTURAL CHOICES THAT MATTER

Beyond the feature mapping, three architectural decisions shape how Paxley delivers the continuous code-trust model described in this article. They are worth surfacing because each addresses a structural concern that scanner-by-scanner additions cannot.

- ▶ **Per-repository pricing** — Security scales with code, not headcount. When AI-assisted development means a 10-engineer team can maintain 500,000 lines of code, billing by developer seat loses its connection to actual risk. Paxley's per-repo model aligns cost with the asset being secured. A 50-developer, 10-repo team pays \$990/month on Starter — typically 70–80% less than per-seat alternatives.
- ▶ **No source-code upload required** — For GitHub-connected repositories, scans can run on customer-controlled GitHub Actions runners. Source code never has to leave customer infrastructure. Paxley receives only normalised security metadata — findings, fingerprints, job IDs, posture data. This directly addresses the source-code custody concern that increasingly impedes adoption of SaaS code security in regulated or IP-sensitive environments.
- ▶ **Twelve engines, one normalised model** — Instead of stitching together SonarQube for quality, Snyk for SCA, GitGuardian for secrets, a separate SBOM tool, and a license scanner, teams get unified findings with consistent severity, fingerprinting, deduplication, and ownership. This is the orchestration-over-sprawl principle that the article advocates, delivered as a single product.

Pillar 3 of the Nucleus Systems Foundation · Code Trust & Secure Digital Delivery

The Paxley Code Security Platform is the technology delivery layer for Nucleus Systems' Code Trust Assurance Framework (NS-CTAF v1.0). Together, the framework and the platform establish measurable, continuously generated trust in software — across the full lifecycle from developer identity and build integrity through to deployment, runtime behaviour, and software-supply-chain assurance. The platform operationalises the article's closing argument: code trust as a continuous capability, not secure development as a phase.

AN HONEST CLOSING NOTE

Paxley is not the only path to the model described in this article. SonarQube, Snyk, Checkmarx, Veracode, GitHub Advanced Security, Semgrep, and other mature vendors can each support parts of the transition. Where Paxley fits is the unified platform for mid-market and fast-growing engineering teams that need broad coverage without enterprise weight, predictable pricing as they scale, and the option to keep source code inside their own infrastructure. Teams running Fortune 500 AppSec programmes with dedicated security engineering and seven-figure budgets are usually better served by established enterprise platforms. Teams needing DAST or IAST today should pair Paxley with a dynamic testing tool, or evaluate Fortify, AppScan, or Veracode. Honesty about fit makes for happier customers and shorter procurement cycles.

The new standard, as the article concludes, is not secure development as a phase, but code trust as a continuous capability. Paxley is built to be exactly that — and to do it for engineering teams that today are forced to choose between fragmented tools, per-seat sticker shock, and source-code custody compromises they would rather not make.

TRY PAXLEY · paxleysoftware.com

Free Community tier for up to 3 private repositories. 14-day Starter trial across all 8 capability areas. 30-day extended trial for teams migrating from Snyk or SonarQube—a product of Nucleus Systems.